

# Практика 5. Типы данных

осень 2026

Андрей Стоян, Илья Колегов

осень 2026

# Летучка 5

Летучка 5 · 7 минут · Ваше ФИ: \_\_\_\_\_

1. Расставьте скобки в типе `String -> Int -> Bool`: \_\_\_\_\_

2. Напишите наиболее общий тип `\x -> (x, x)`: \_\_\_\_\_

3. Сделайте один  $\beta$ -шаг:  $(\lambda x. x\ a)\ b \rightsquigarrow$  \_\_\_\_\_

4. Дано `fac 0 = 1; fac n = n * fac (n - 1)`. Раскройте только вызов, арифметику не считайте: `fac 3`  $\rightsquigarrow$  \_\_\_\_\_

5. Список `[10, 20, 30]` сопоставили с образцом `x : xs`. Чему равно `xs`? \_\_\_\_\_

# Конструкторы и сопоставление

# Тип или конструктор?

✎ Даны объявления. Напишите типы `CatD`, `Dog`, `Dog "barsik"` и `Parrot`.

```
data CatT = CatD String Int
data Pet  = Cat CatT | Dog String Int | Parrot
```

## ⚡Решение.

```
ghci> :t CatD
CatD :: String -> Int -> CatT
ghci> :t Dog
Dog :: String -> Int -> Pet
ghci> :t Dog "barsik"
Dog "barsik" :: Int -> Pet
ghci> :t Parrot
Parrot :: Pet
```

- `CatT`, `Pet` — типы; `CatD`, `Cat`, `Dog`, `Parrot` — конструкторы данных
- Конструктор можно применить частично, как функцию

# Разобрать то, что сконструировали

```
showPet :: Pet -> String
showPet pet = case pet of
  Parrot -> "Parrot"
  Dog name _ -> "Dog: " ++ name
  Cat (CatD name _) -> "Cat: " ++ name
```

✎ **Устно:** Какую ветку выберет `showPet (Cat (CatD "murka" 2))`? Какие имена свяжет образец?

⚡ **Решение.**

Последнюю: `name = "murka"`, возраст игнорируется. Результат — `"Cat: murka"`.

- **Δ Сопоставление с образцом** разбирает значение по конструктору
- Вложенный образец разбирает и внешнюю обёртку `Cat`, и внутреннюю `CatD`

# Возраст: что делать с попугаем?

✎ Закончите функцию, чтобы она обрабатывала каждый вариант `Pet`. Возраст попугая в нашем типе не хранится.

```
age :: Pet -> Maybe Int
age pet = case pet of
  Dog _ years -> Just years
  Cat (CatD _ years) -> Just years
  -- допишите недостающую ветку
```

## ⚡Решение.

```
Parrot -> Nothing
```

- **Δ Полнота сопоставления:** учтены все конструкторы
- `Nothing` — нет данных о возрасте; `Just 0` — известный возраст, равный нулю

# Рекурсивное дерево

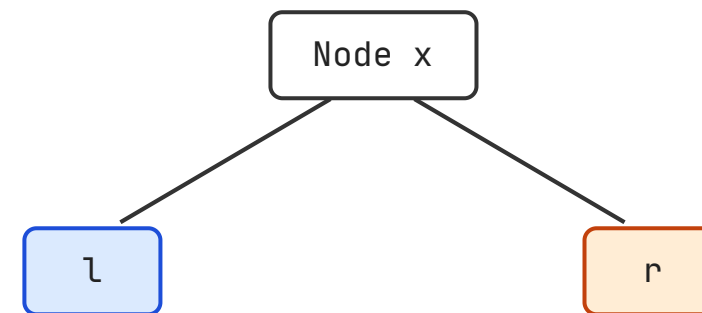
# Функция следует за структурой типа

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

Два конструктора → две ветки

```
height :: Tree a -> Int
height t = case t of
  Leaf _ -> 1
  Node l _ r ->
    1 + max (height l) (height r)
```

Два поддеревя → два вызова



# Предскажите высоту

✎ Запишите раскрытие `height t` до числового результата. Высота листа — 1.

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
t = Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))
height (Leaf _) = 1
height (Node l _ r) = 1 + max (height l) (height r)
```

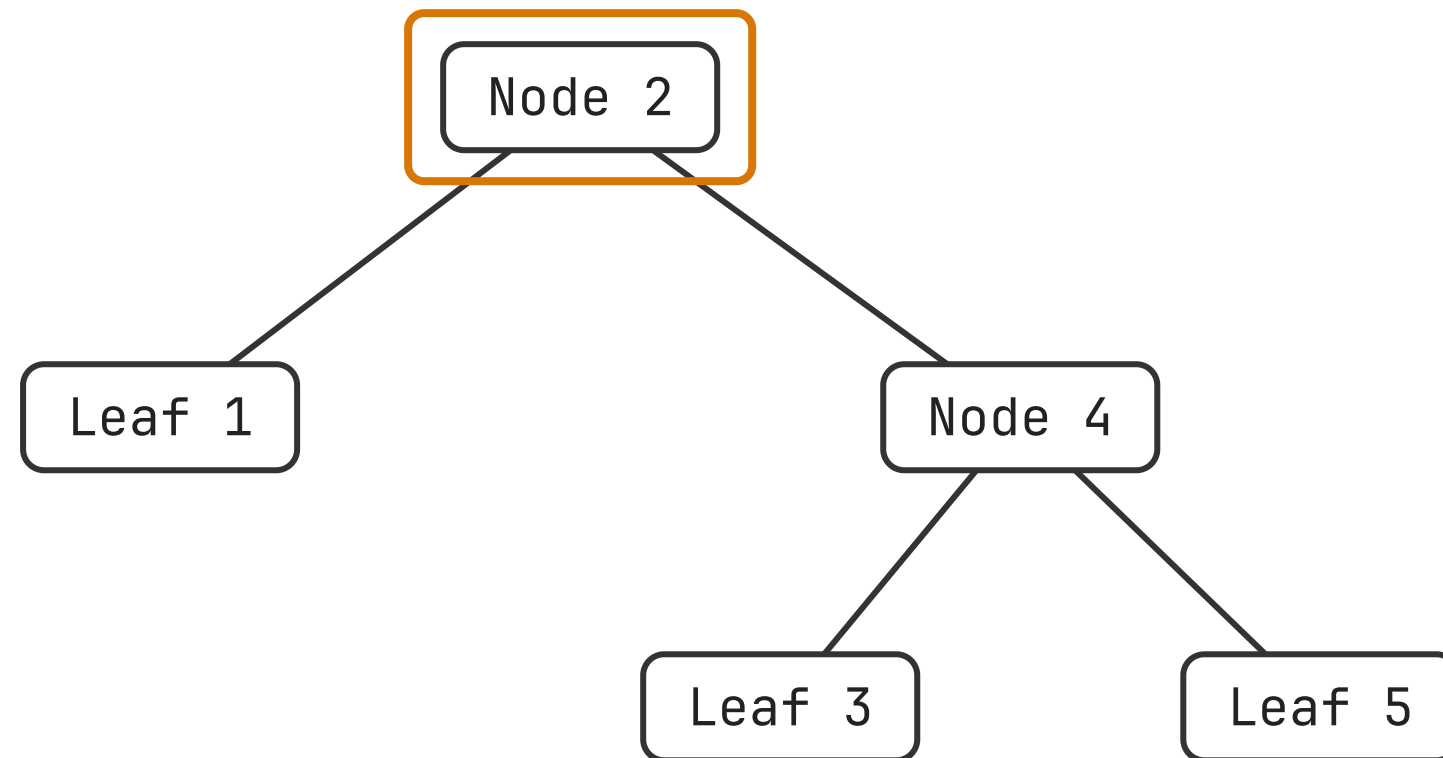
# Высота: спуск и возврат

В **Node** два рекурсивных вызова. Сначала получим результат слева.

```
1 + max (height l) (height r)
```

ВЫЗОВЫ И РЕЗУЛЬТАТЫ

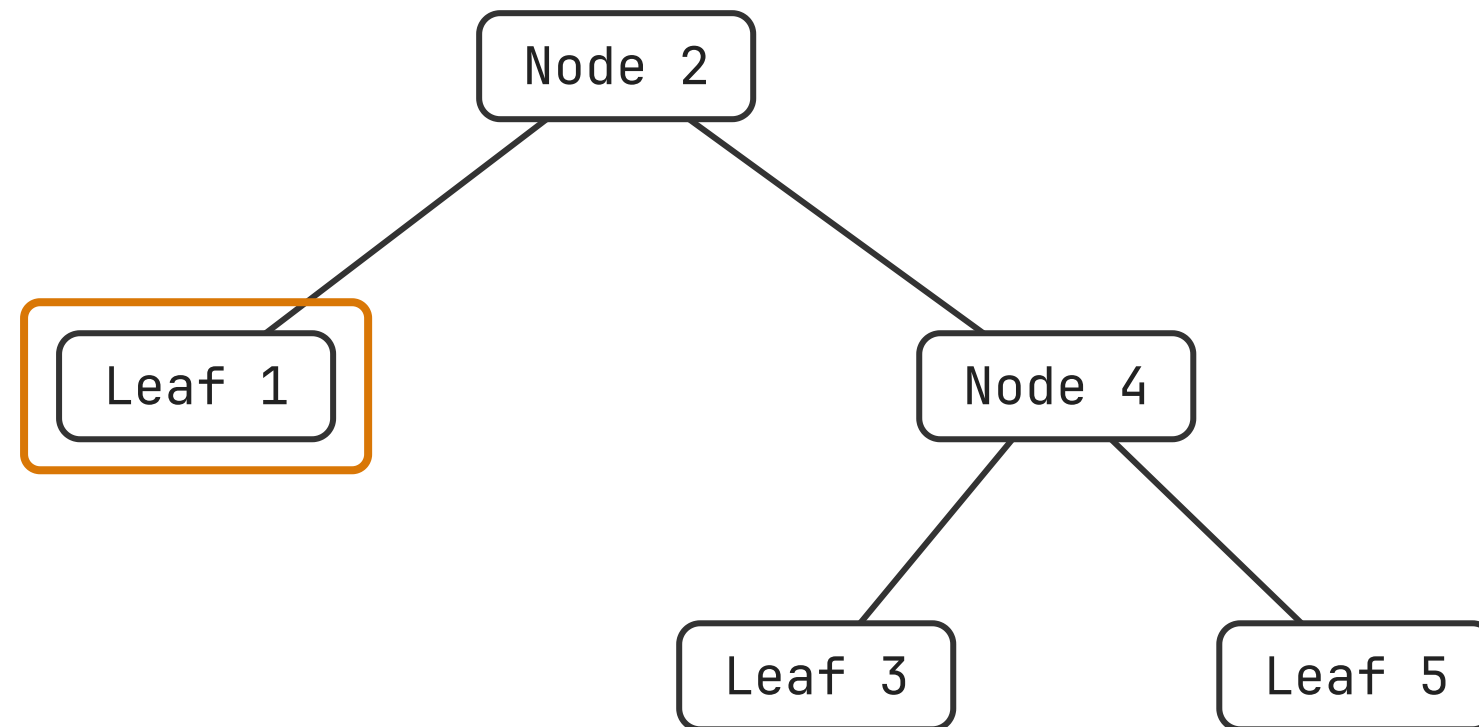
**Node 2: ждём l и r**



# Высота: спуск и возврат

Спускаемся в левое поддерево: встретили **Leaf**.

`height (Leaf 1)`



ВЫЗОВЫ И РЕЗУЛЬТАТЫ

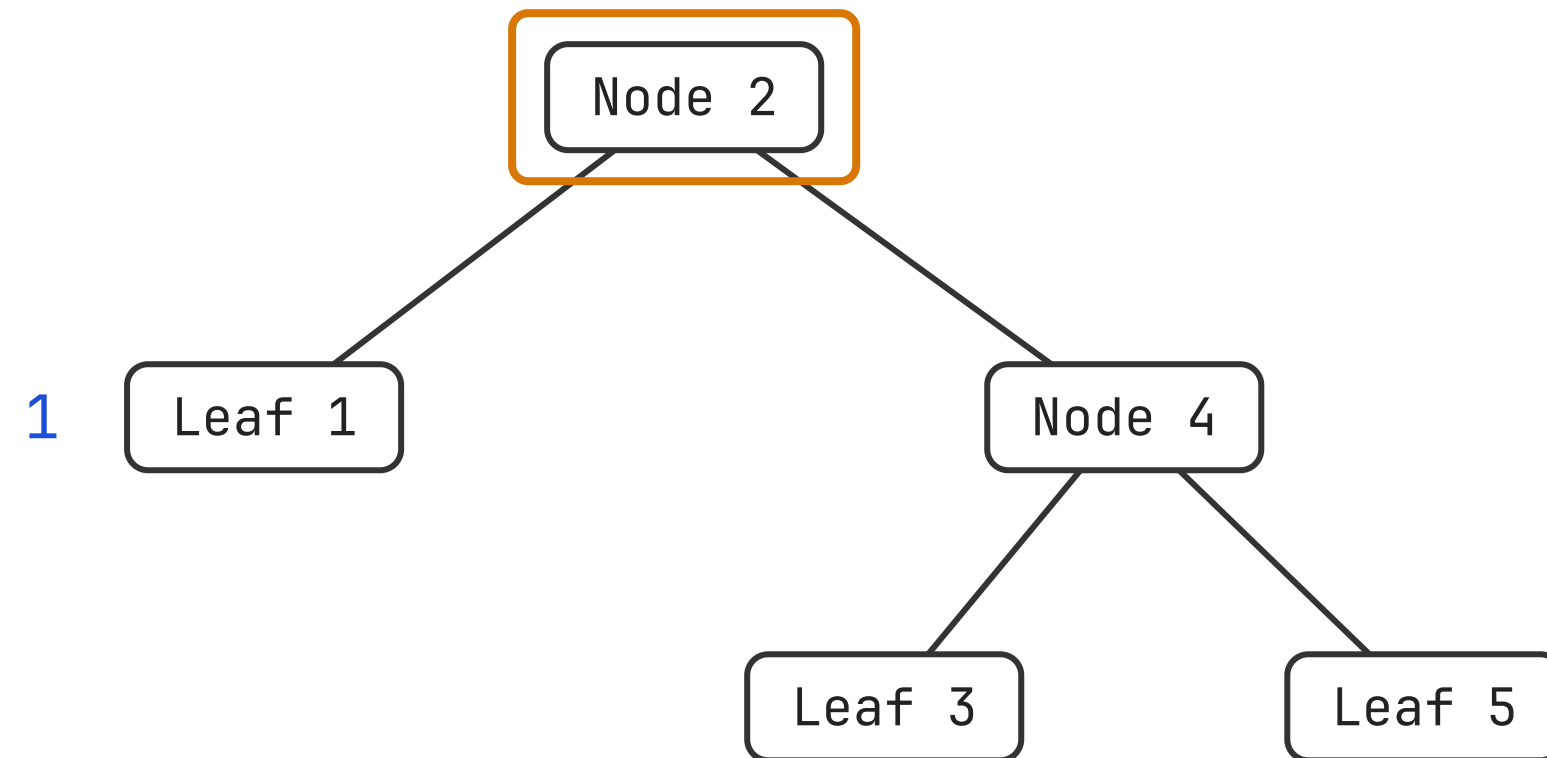
Node 2: ждём `l` и `r`

↓ **Leaf 1: база**

# Высота: спуск и возврат

Возвращаем **1** родителю. Правый результат ещё неизвестен.

$1 + \max 1 (\text{height } r)$



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём  $l$  и  $r$

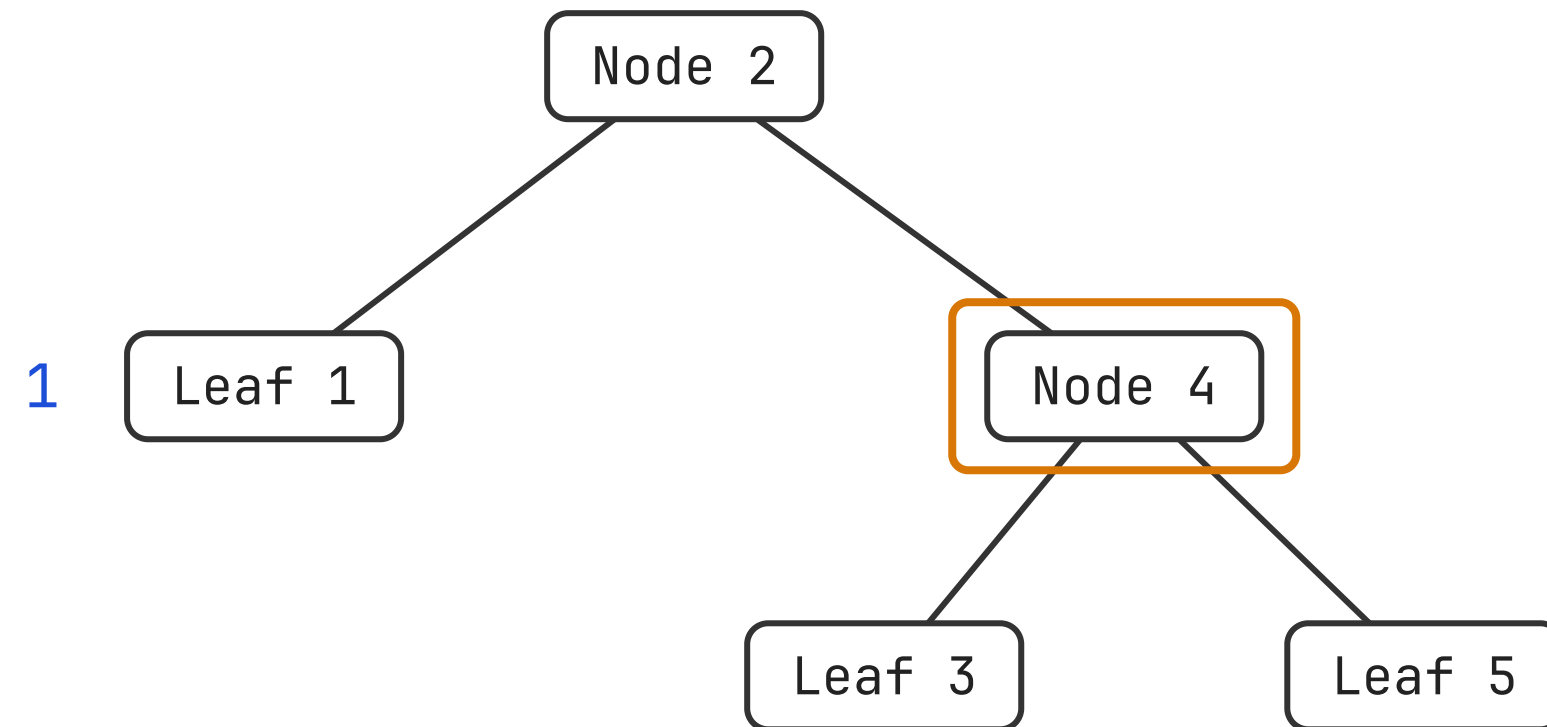
↓ Leaf 1: база

↑ **слева вернулась 1**

# Высота: спуск и возврат

Спускаемся вправо. Здесь снова **Node** с двумя поддеревьями.

$1 + \max(\text{height } r_l) (\text{height } r_r)$



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём  $l$  и  $r$

↓ Leaf 1: база

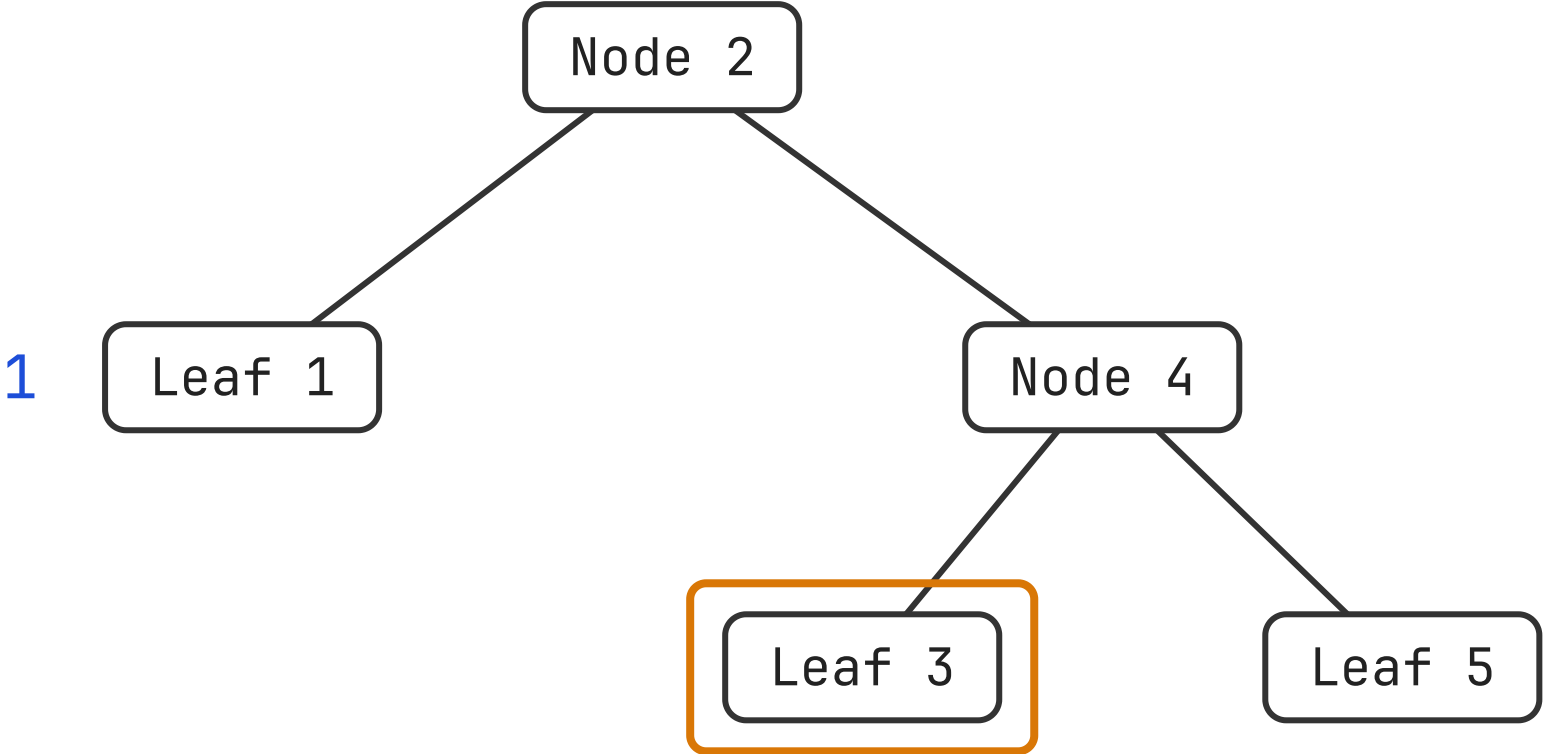
↑ слева вернулась 1

↓ Node 4: ждём  $r_l$ ,  $r_r$

# Высота: спуск и возврат

Левый ребёнок этого узла — лист. Его высота тоже 1.

height (Leaf 3)



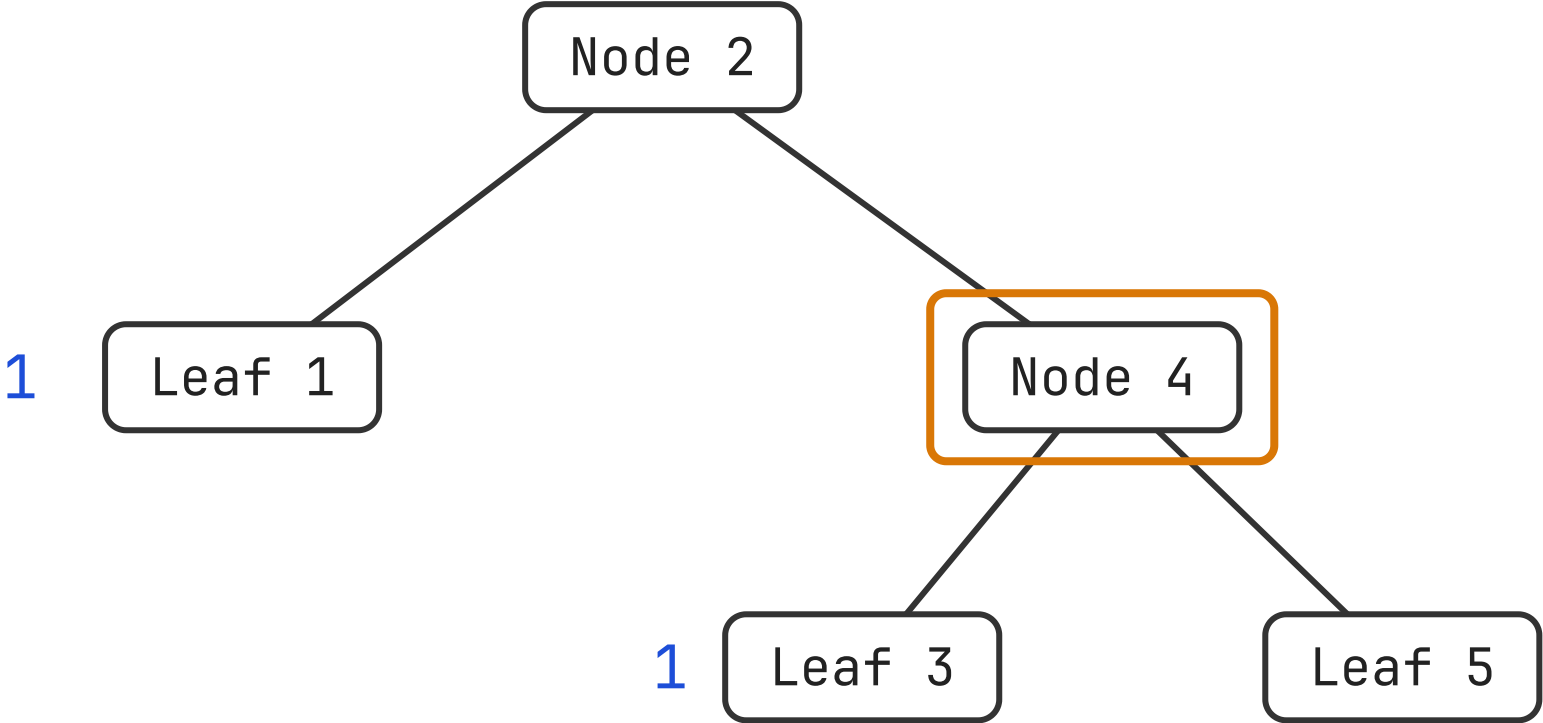
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём l и r
- ↓ Leaf 1: база
- ↑ слева вернулась 1
- ↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>
- ↓ Leaf 3: база

# Высота: спуск и возврат

Возвращаемся к Node 4: слева уже 1, справа ещё вызов.

```
1 + max 1 (height rr)
```



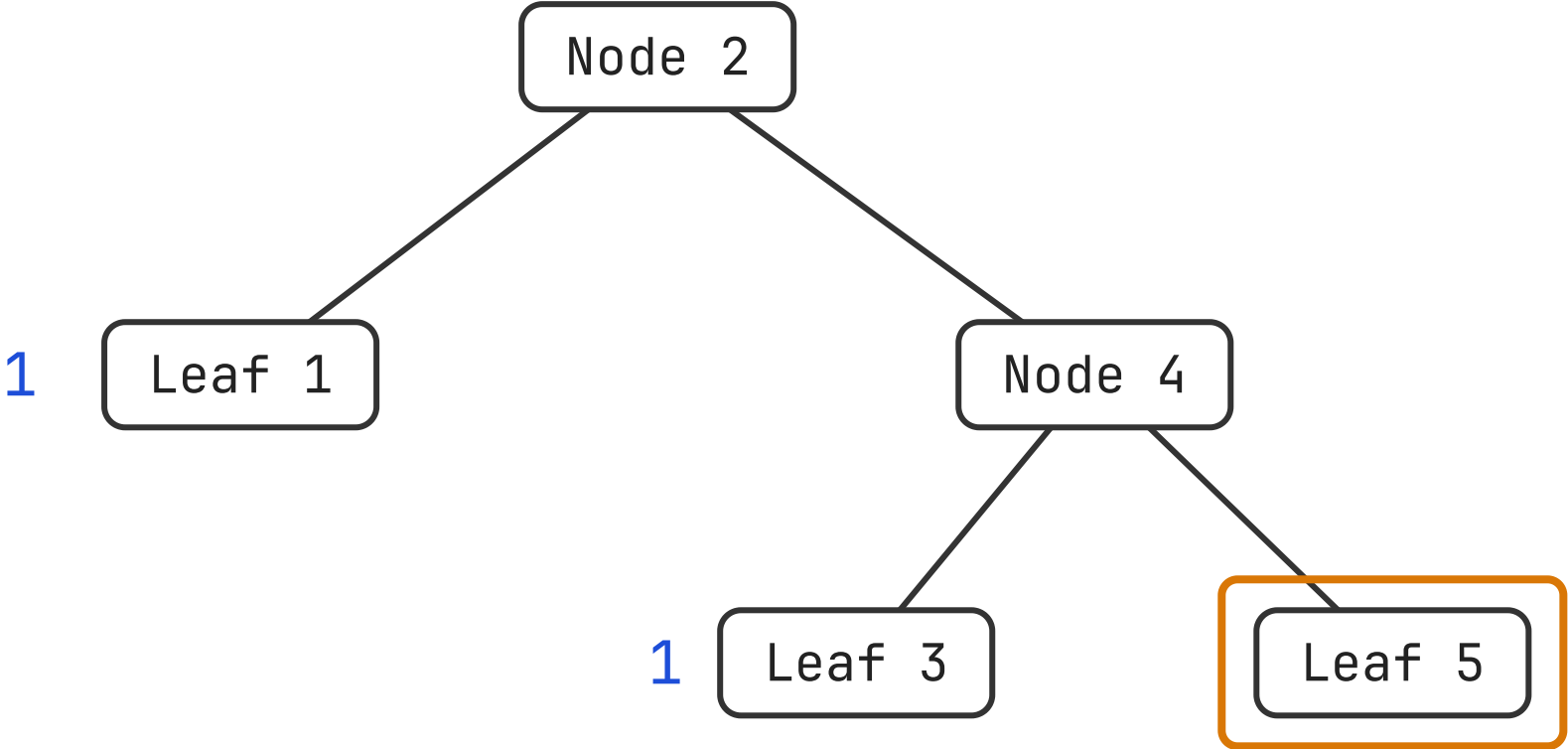
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём l и r
- ↓ Leaf 1: база
- ↑ слева вернулась 1
- ↓ Node 4: ждём r<sub>l</sub>, rr
- ↓ Leaf 3: база
- ↑ от Leaf 3 вернулась 1

# Высота: спуск и возврат

Спускаемся к последнему листу.

height (Leaf 5)



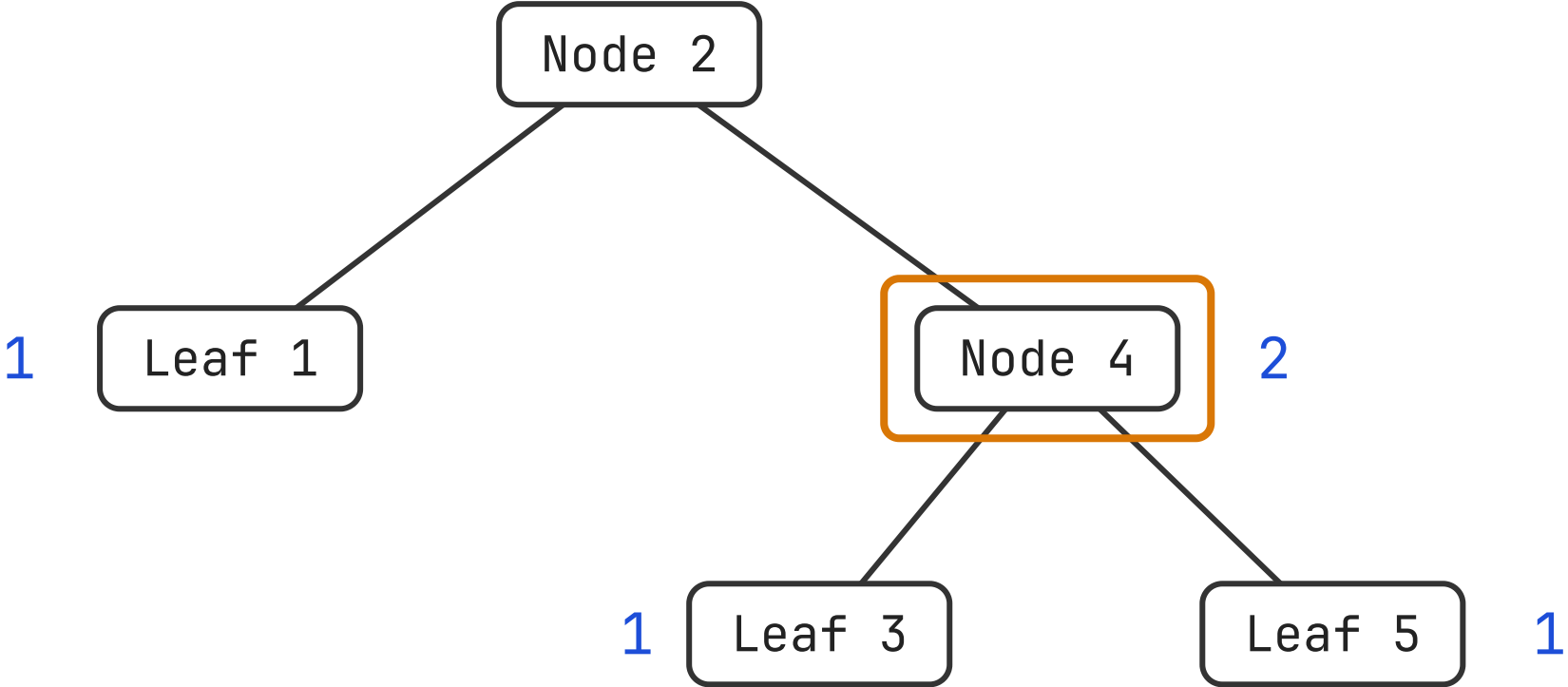
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём  $\ell$  и  $r$
- ↓ Leaf 1: база
- ↑ слева вернулась 1
- ↓ Node 4: ждём  $r\ell$ ,  $rr$
- ↓ Leaf 3: база
- ↑ от Leaf 3 вернулась 1
- ↓ Leaf 5: база

# Высота: спуск и возврат

Оба результата готовы: высота правого поддерева равна 2.

$1 + \max 1 \ 1 = 2$



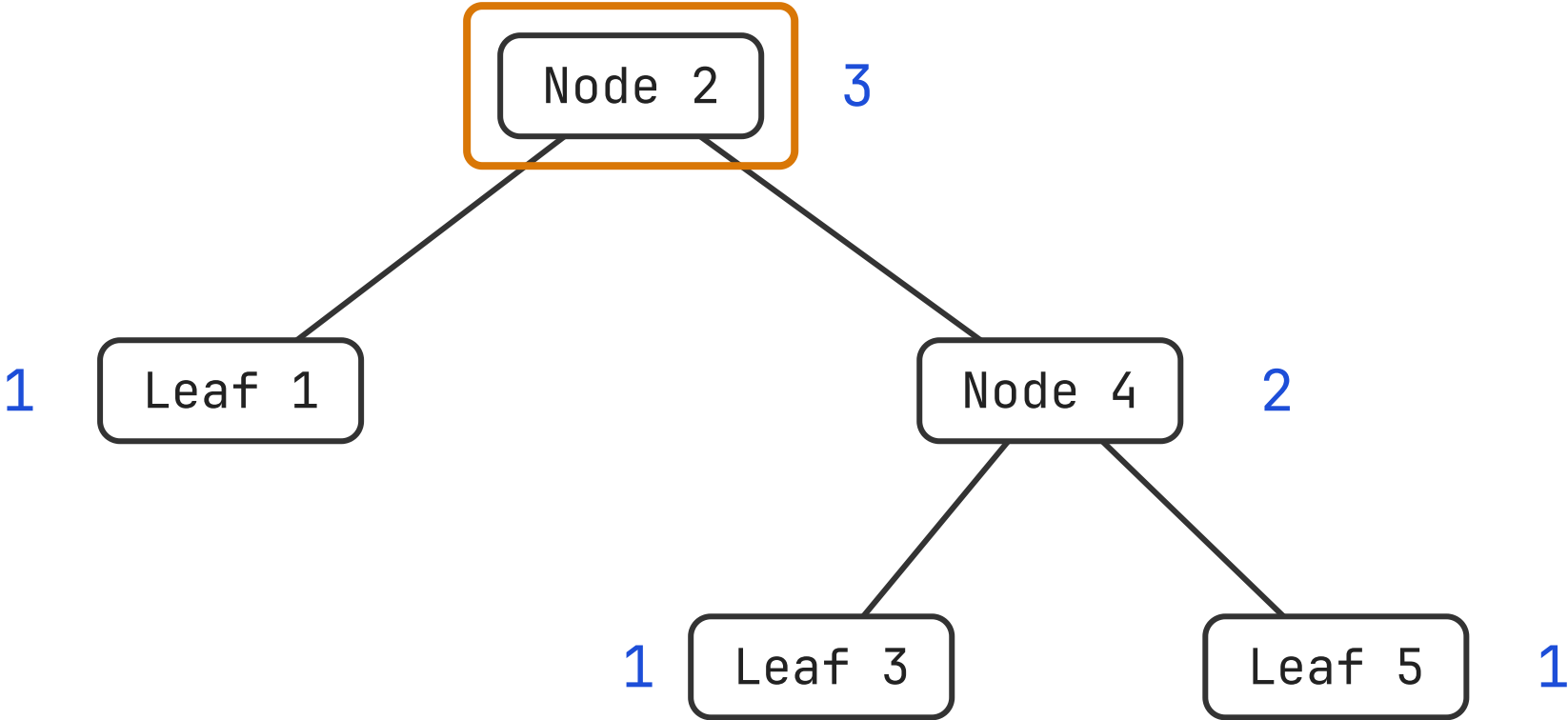
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём l и r
- ↓ Leaf 1: база
- ↑ слева вернулась 1
- ↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>
- ↓ Leaf 3: база
- ↑ от Leaf 3 вернулась 1
- ↓ Leaf 5: база
- ↑ Node 4: **результат 2**

# Высота: спуск и возврат

Возвращаем 2 корню. Всё дерево имеет высоту 3.

$1 + \max 1\ 2 = 3$



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём л и r
- ↓ Leaf 1: база
- ↑ слева вернулась 1
- ↓ Node 4: ждём rл, rr
- ↓ Leaf 3: база
- ↑ от Leaf 3 вернулась 1
- ↓ Leaf 5: база
- ↑ Node 4: результат 2
- ↑ **корень: результат 3**

# Теперь сумма: тот же каркас

✎ Заполните три пропуска и найдите сумму значений того же дерева **t**.

```
sumTree :: Tree Int -> Int
sumTree t = case t of
  Leaf x -> ___
  Node l x r -> ___ + x + ___
```

⚡ **Решение.**

```
sumTree t = case t of
  Leaf x -> x
  Node l x r -> sumTree l + x + sumTree r
```

Сумма — **15**.

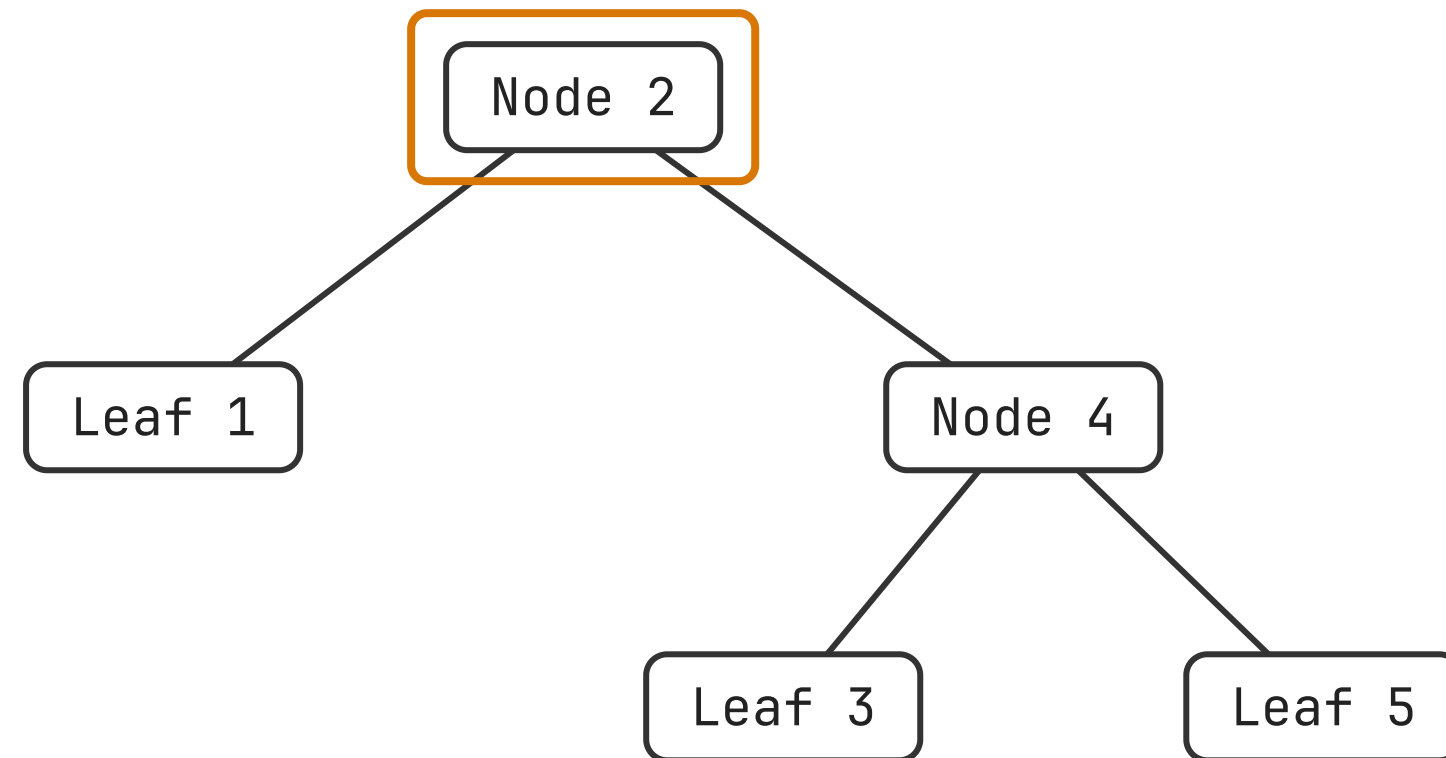
# Сумма: тот же спуск, другие результаты

Та же структура вызовов. Теперь учитываем и значение в каждом узле.

```
sumTree l + 2 + sumTree r
```

ВЫЗОВЫ И РЕЗУЛЬТАТЫ

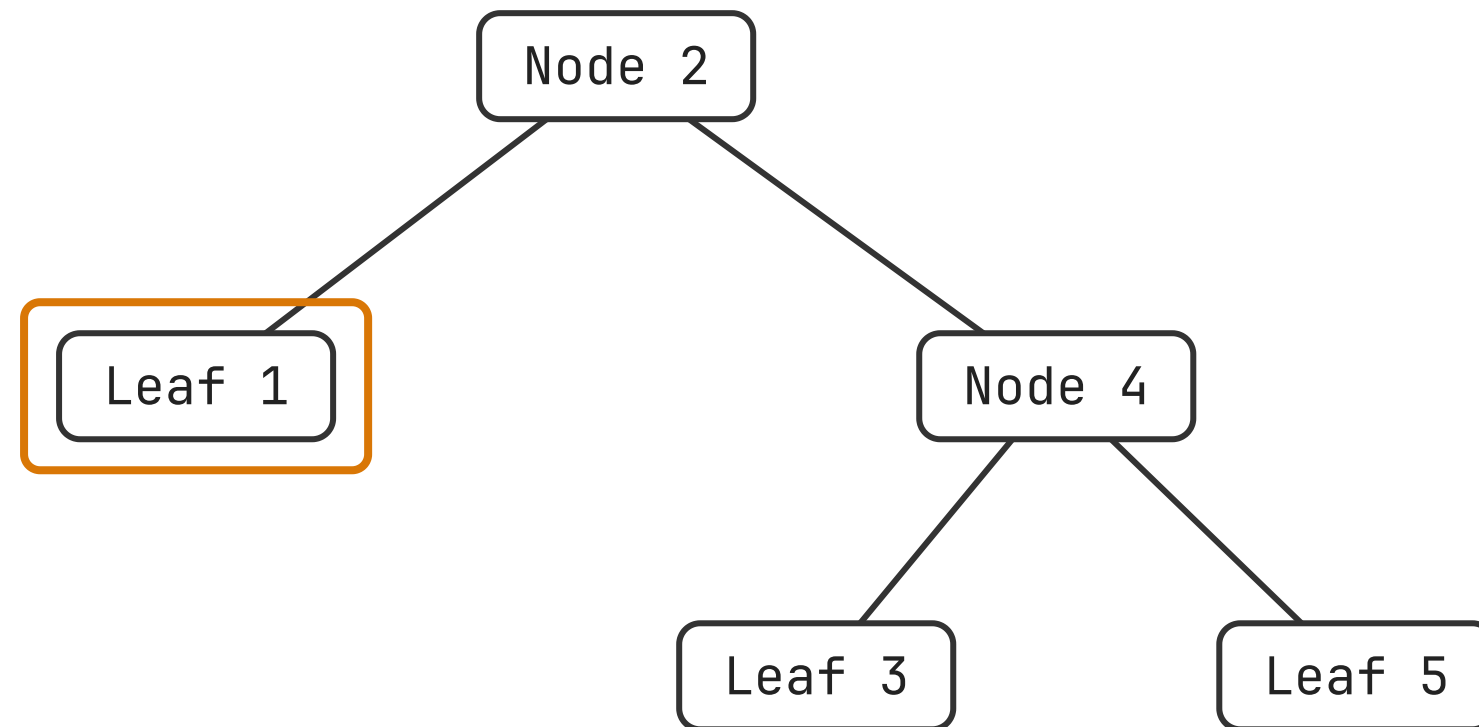
Node 2: ждём l и r



# Сумма: тот же спуск, другие результаты

Спускаемся влево. Лист возвращает своё значение.

`sumTree (Leaf 1)`



ВЫЗОВЫ И РЕЗУЛЬТАТЫ

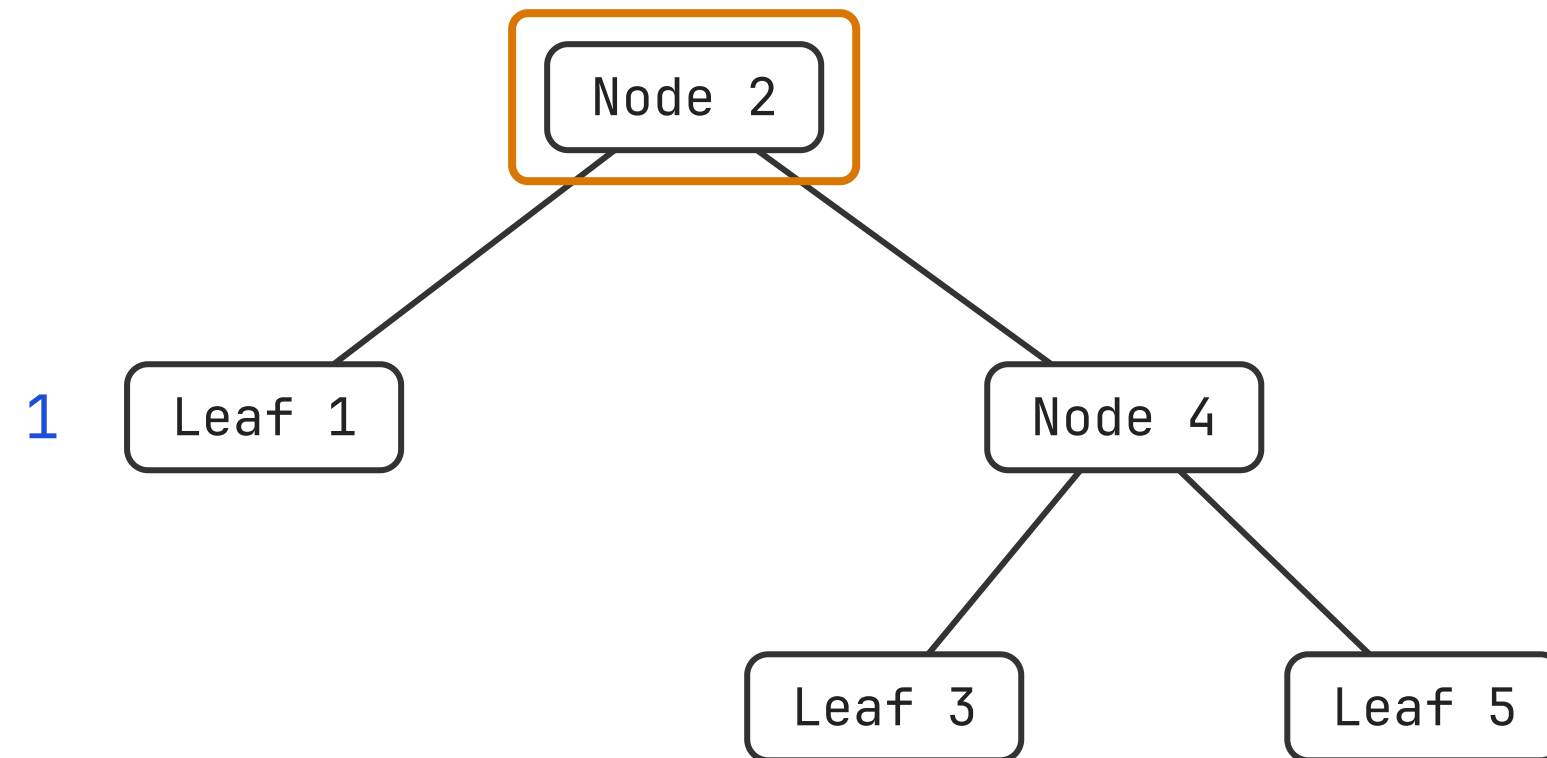
Node 2: ждём `l` и `r`

↓ Leaf 1: **возвращает 1**

# Сумма: тот же спуск, другие результаты

Левый результат вернулся. Корню ещё нужна сумма справа.

$1 + 2 + \text{sumTree } r$



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём  $l$  и  $r$

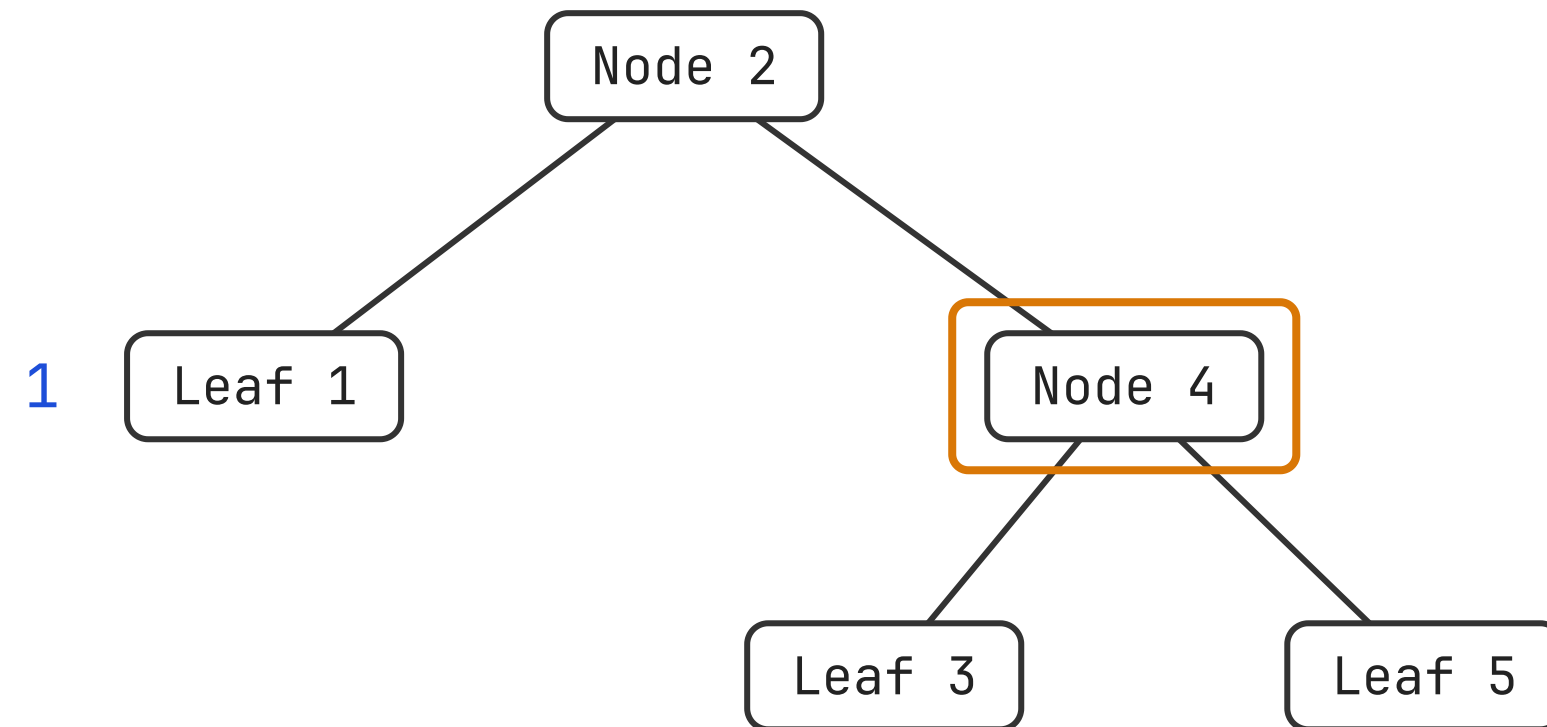
↓ Leaf 1: возвращает 1

↑ **слева вернулась 1**

# Сумма: тот же спуск, другие результаты

В правом поддереве снова тот же каркас.

```
sumTree rл + 4 + sumTree rr
```



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём л и r

↓ Leaf 1: возвращает 1

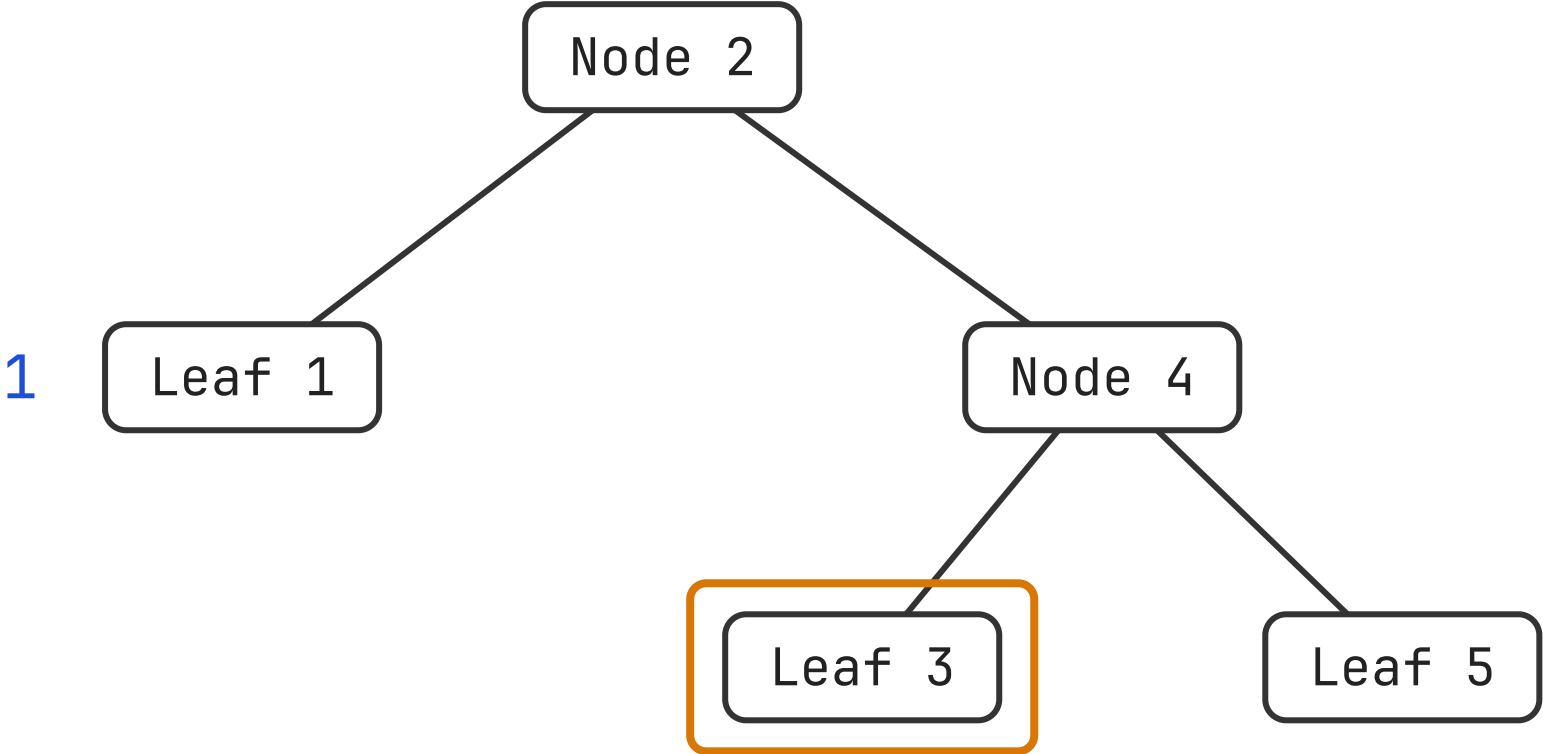
↑ слева вернулась 1

↓ Node 4: ждём rл, rr

# Сумма: тот же спуск, другие результаты

Левый лист возвращает 3.

sumTree (Leaf 3)



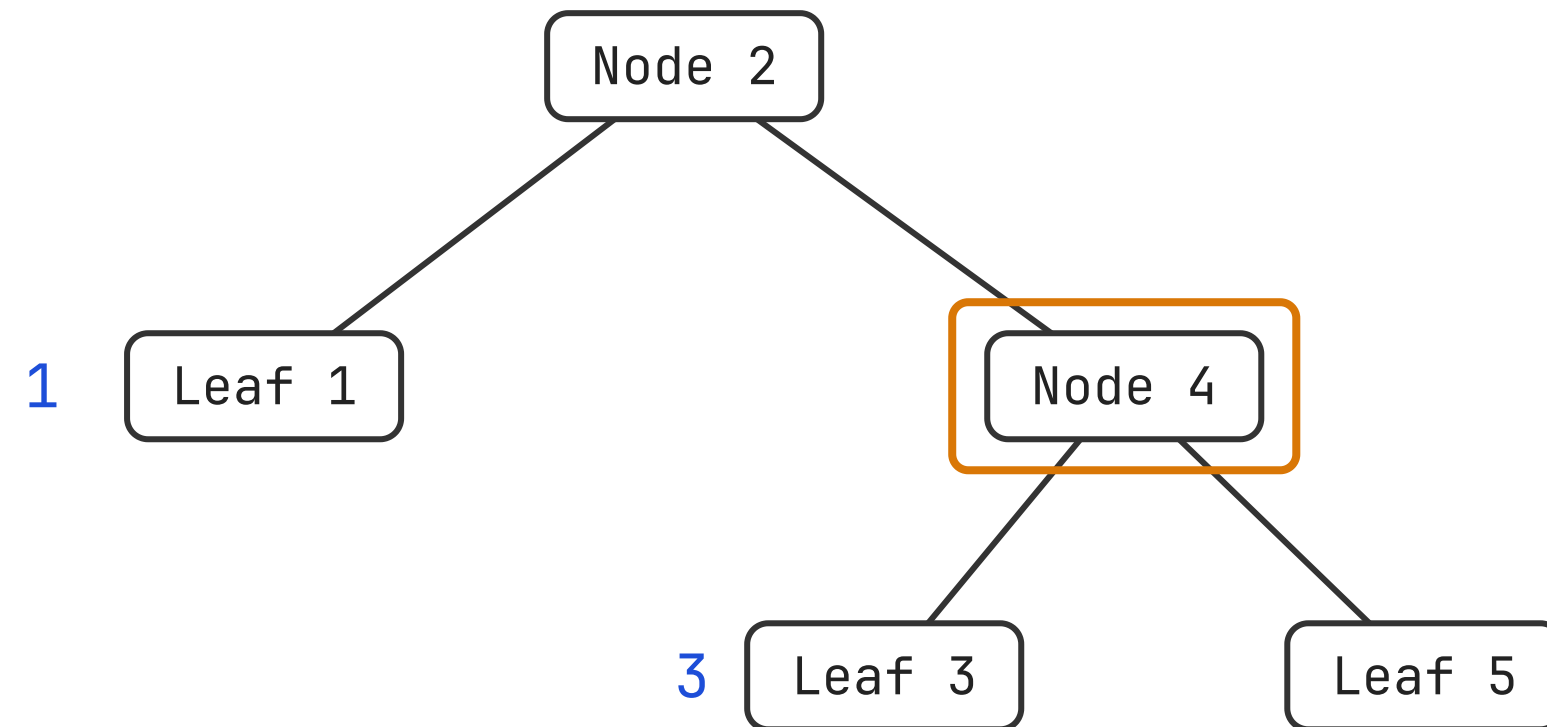
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём l и r  
↓ Leaf 1: возвращает 1  
↑ слева вернулась 1  
↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>  
↓ Leaf 3: **возвращает 3**

# Сумма: тот же спуск, другие результаты

Возвращаемся к родителю: остался один рекурсивный вызов.

`3 + 4 + sumTree rr`



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём  $\ell$  и  $r$

↓ Leaf 1: возвращает 1

↑ слева вернулась 1

↓ Node 4: ждём  $r\ell$ ,  $rr$

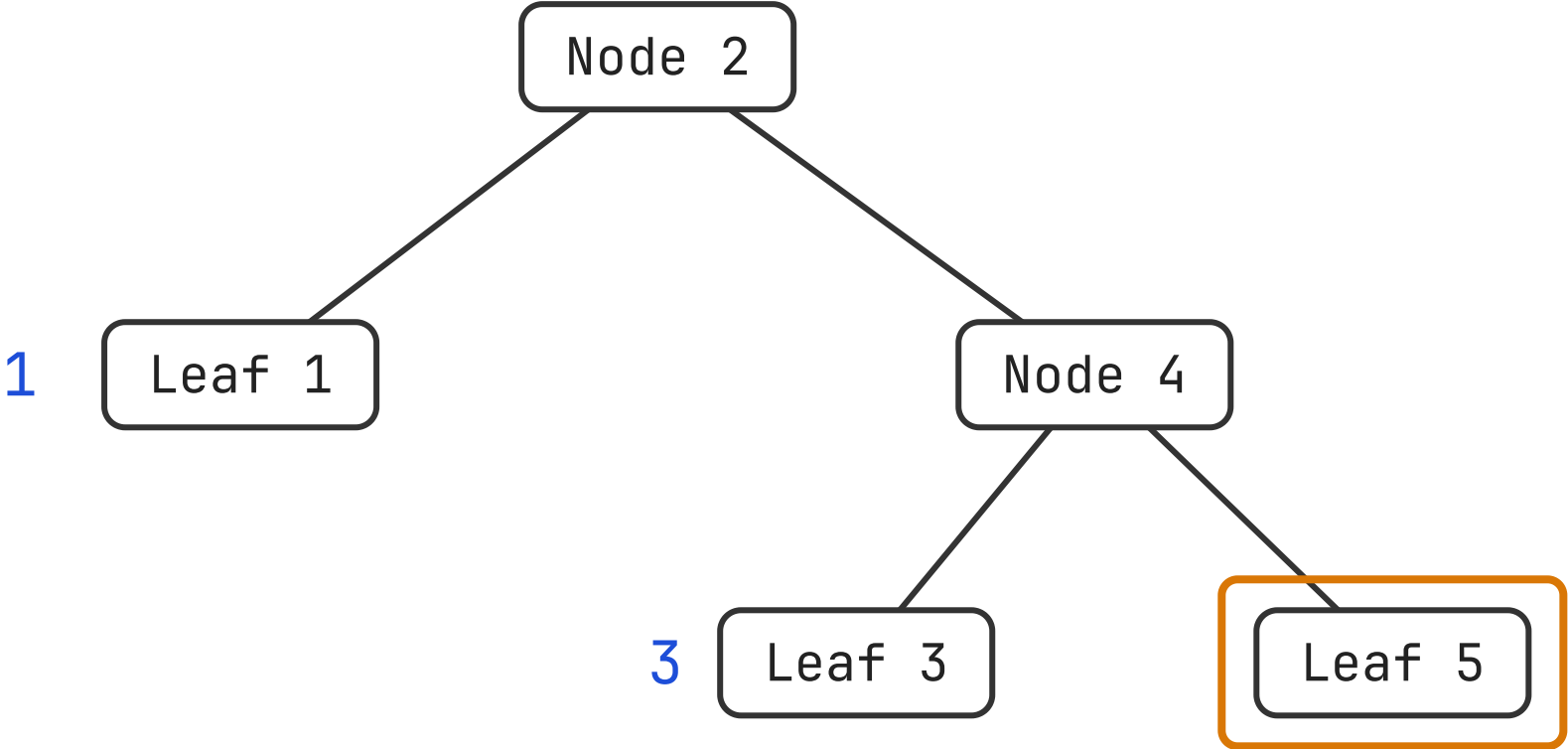
↓ Leaf 3: возвращает 3

↑ **от Leaf 3 вернулась 3**

# Сумма: тот же спуск, другие результаты

Последний лист возвращает 5.

sumTree (Leaf 5)



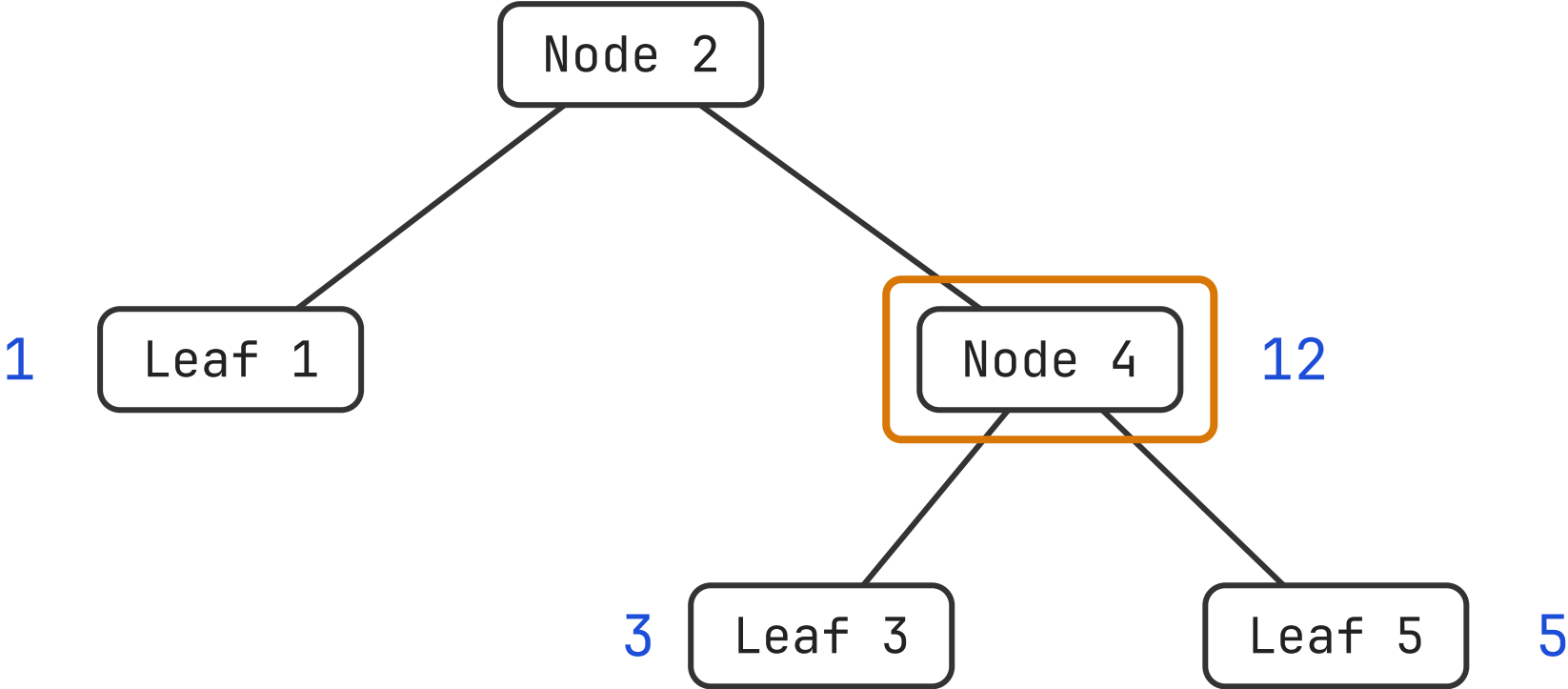
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

- Node 2: ждём l и r
- ↓ Leaf 1: возвращает 1
- ↑ слева вернулась 1
- ↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>
- ↓ Leaf 3: возвращает 3
- ↑ от Leaf 3 вернулась 3
- ↓ Leaf 5: возвращает 5

# Сумма: тот же спуск, другие результаты

У правого узла готовы все компоненты суммы: 3 + 4 + 5 = 12.

3 + 4 + 5 = 12



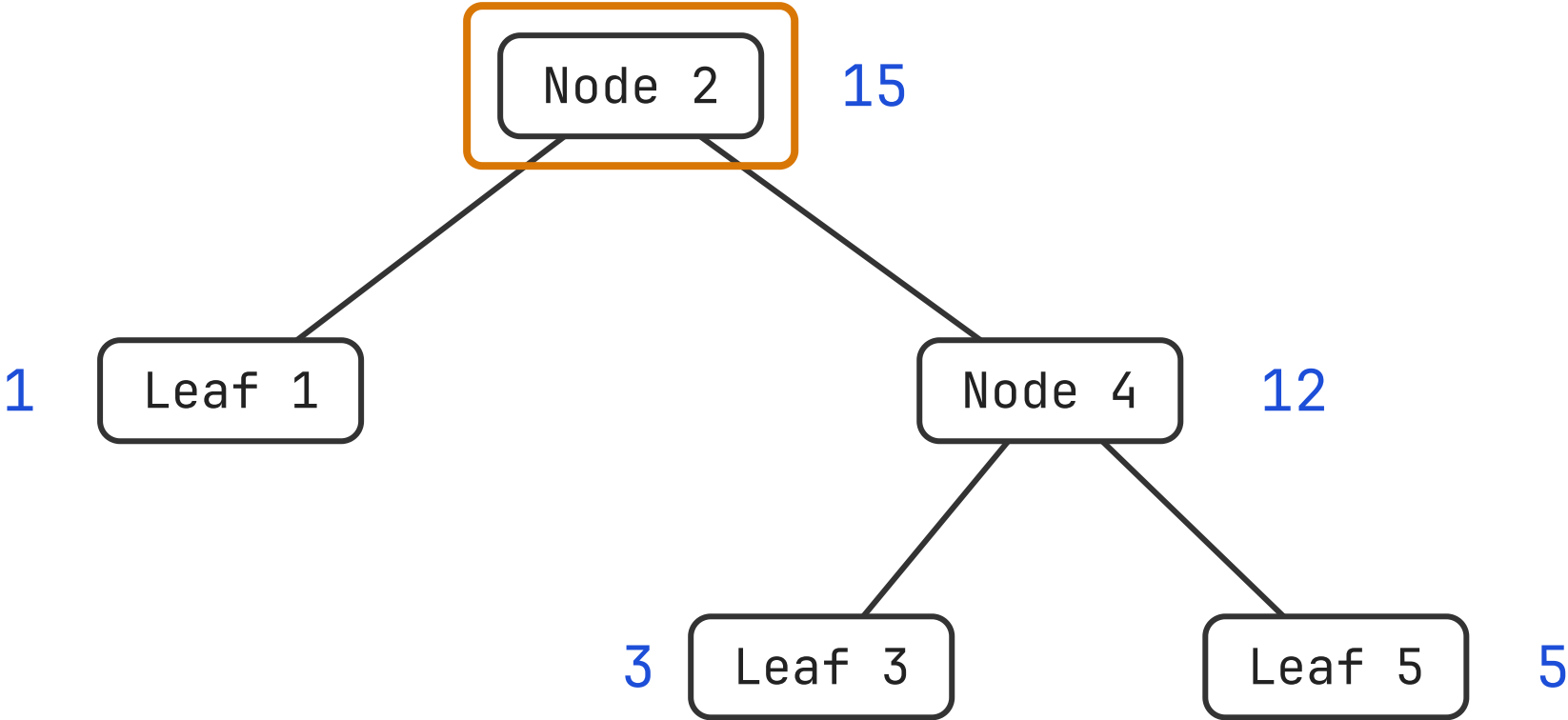
## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём l и r  
↓ Leaf 1: возвращает 1  
↑ слева вернулась 1  
↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>  
↓ Leaf 3: возвращает 3  
↑ от Leaf 3 вернулась 3  
↓ Leaf 5: возвращает 5  
↑ Node 4: **результат 12**

# Сумма: тот же спуск, другие результаты

Возвращаем 12 корню: итоговая сумма равна 15.

1 + 2 + 12 = 15



## ВЫЗОВЫ И РЕЗУЛЬТАТЫ

Node 2: ждём l и r  
↓ Leaf 1: возвращает 1  
↑ слева вернулась 1  
↓ Node 4: ждём r<sub>l</sub>, r<sub>r</sub>  
↓ Leaf 3: возвращает 3  
↑ от Leaf 3 вернулась 3  
↓ Leaf 5: возвращает 5  
↑ Node 4: результат 12  
**↑ корень: результат 15**

# Полиморфизм и кайнды

# Меняем значения, сохраняем дерево

✎ Заполните пропуски. Функция `f` может менять тип содержимого, а форма дерева должна сохраниться.

```
transform :: (a -> b) -> Tree a -> Tree b
transform f t = case t of
  Leaf x -> Leaf ___
  Node l x r -> Node ___ ___ ___
```

## ⚡Решение.

```
transform f t = case t of
  Leaf x -> Leaf (f x)
  Node l x r -> Node (transform f l) (f x) (transform f r)
```

- Схема рекурсии прежняя; результаты теперь собираются обратно в дерево
- Высота не меняется: `height (transform f t) == height t`

# Терм → тип → кайнд

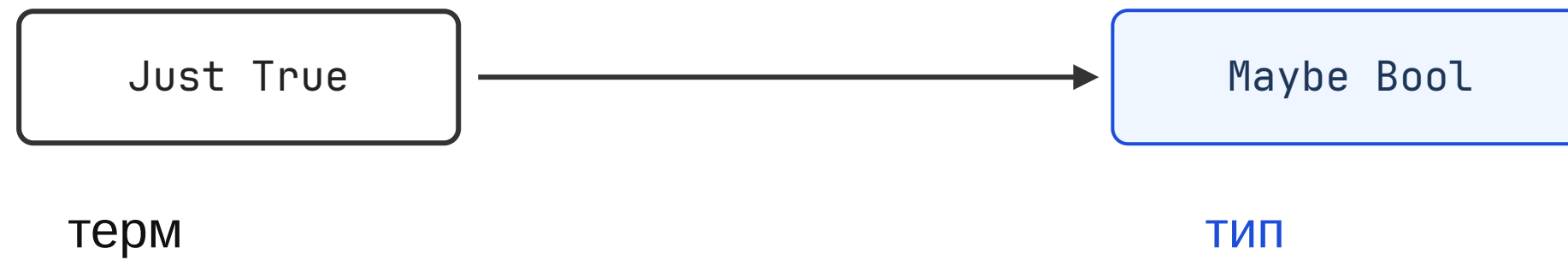
Начинаем с терма: `Just True` — выражение, которое можно вычислить.

Just True

терм

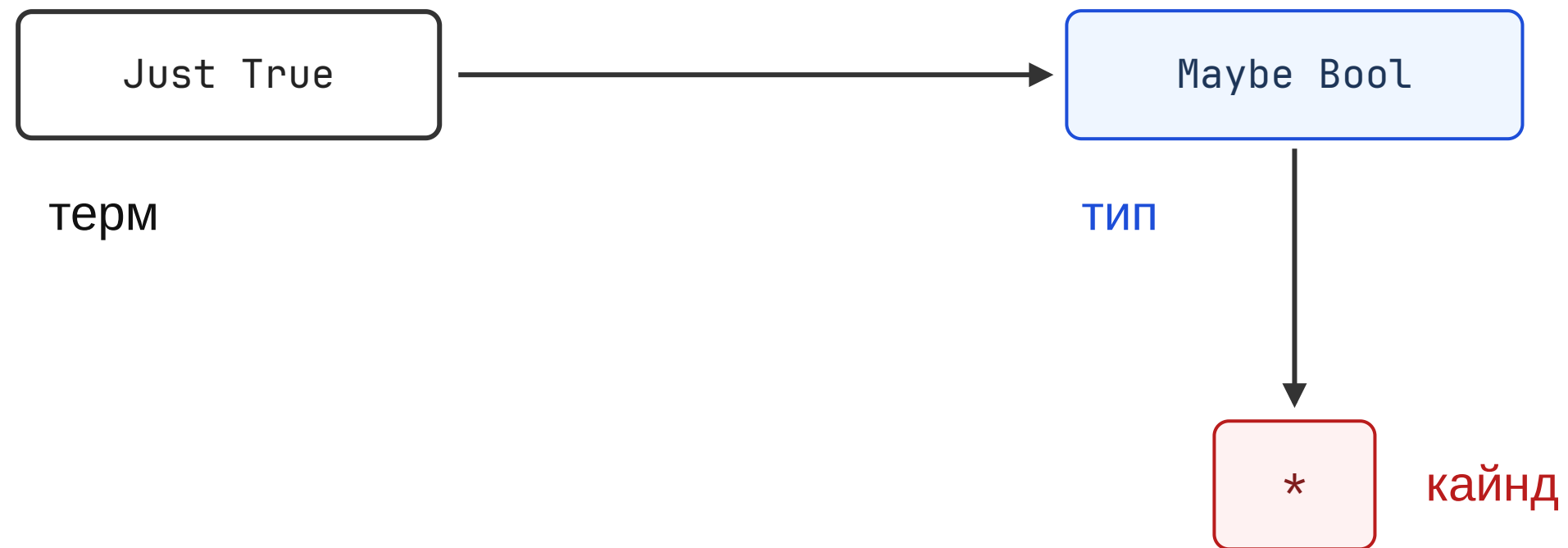
# Терм → тип → кайнд

Тип терма описывает результат: `Just True :: Maybe Bool`.



# Терм → тип → кайнд

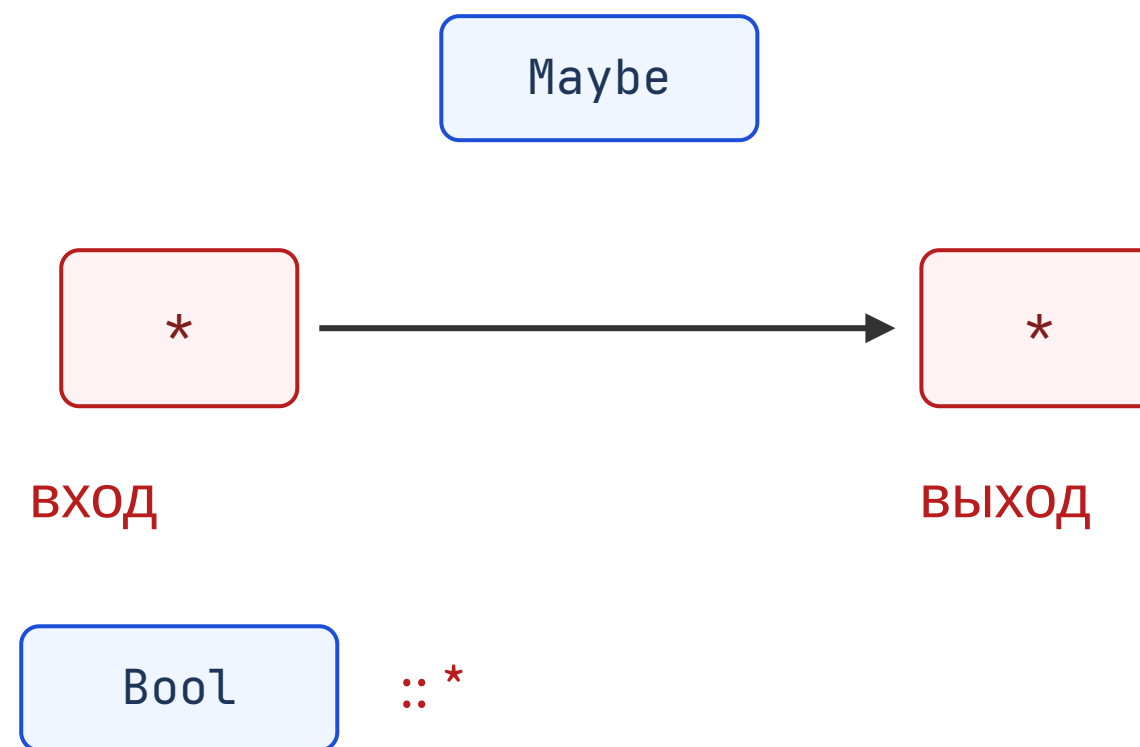
У типа тоже есть характеристика: `Maybe Bool :: *`. Её называют **кайндом**.



# Кайнд: что принимает конструктор типа

`Maybe :: * -> *`: принимает обычный тип и даёт обычный тип.

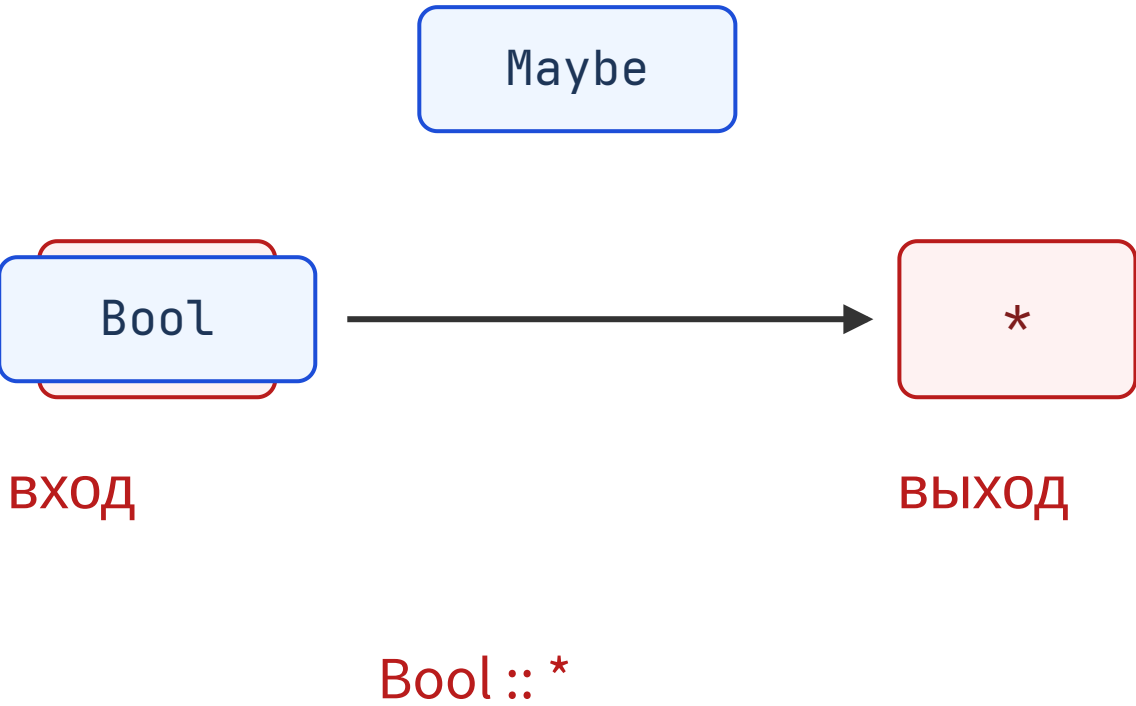
`Maybe Bool`



# Кайнд: что принимает конструктор типа

Тип Bool занимает входной слот: кайнды совпали.

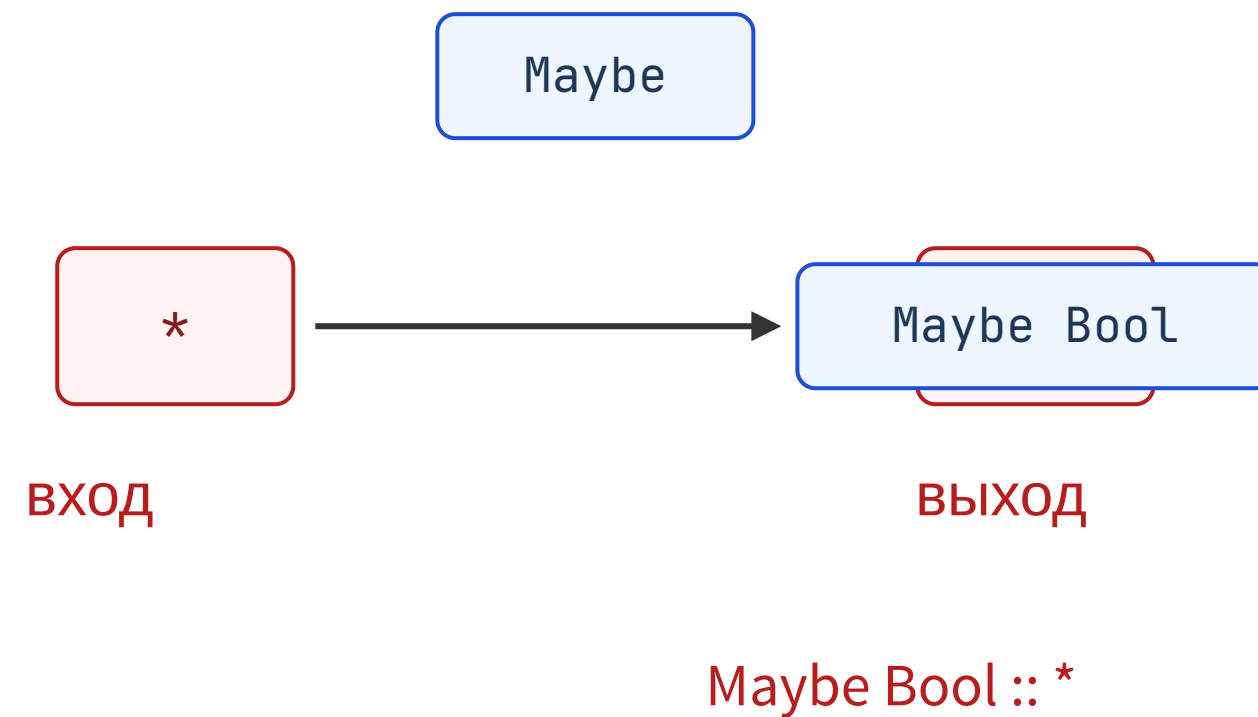
Maybe Bool



# Кайнд: что принимает конструктор типа

Получаем `Maybe Bool :: *`. Стрелка в кайнде использована применением.

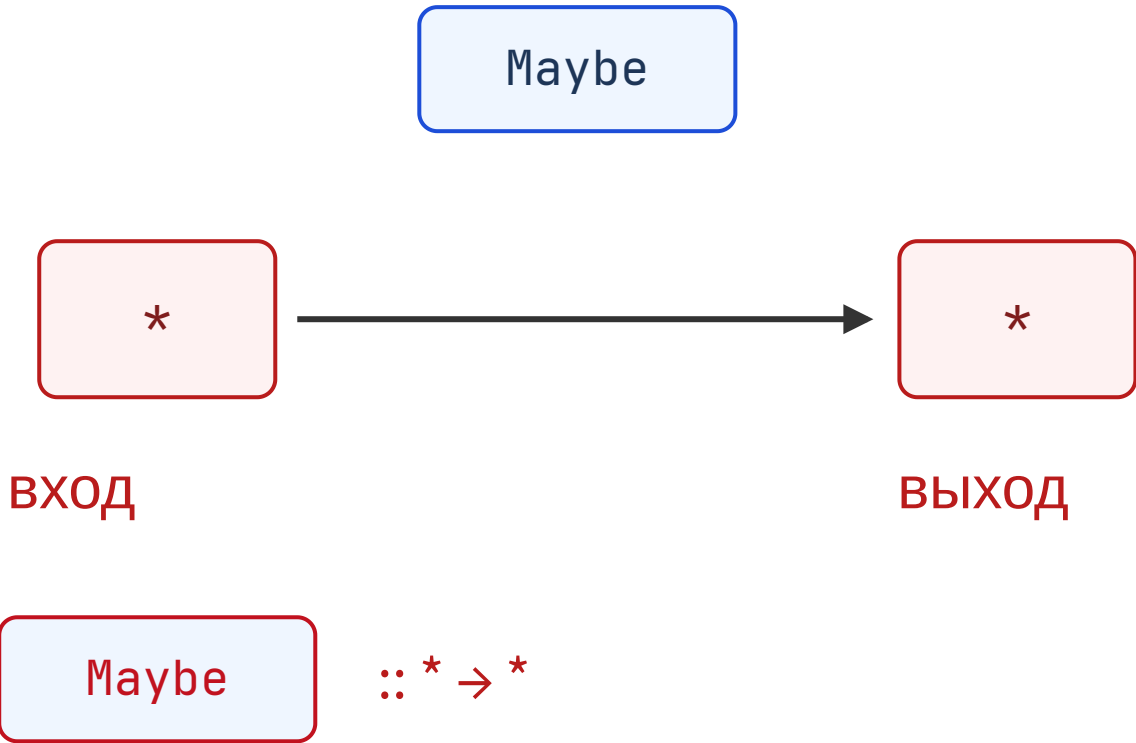
`Maybe Bool :: *`



# Кайнд: что принимает конструктор типа

А `Maybe Maybe` не подходит: аргумент имеет кайнд `* -> *`, вход требует `*`.

`Maybe Maybe`



# Предскажите кайнд

✎ **Устно:** Определите кайнды `Tree`, `Tree Bool`, `Either String` и `Either String Int`. Затем проверим через `:k`.

## ⚡Решение.

```
ghci> :kind Tree
Tree :: * -> *
ghci> :kind Tree Bool
Tree Bool :: *
ghci> :kind Either String
Either String :: * -> *
ghci> :kind Either String Int
Either String Int :: *
```

- `:t` спрашивает тип **терма**; `:k` спрашивает кайнд **типа**
- `Node` — конструктор данных; `Tree` — конструктор типа

# Списки

# Список: две ветки, одно поддерево

```
-- Схематически: встроенный список устроен так
-- data List a = Nil | Cons a (List a)
[]          -- пустой список
x : xs     -- голова и хвост
```

 Напишите безопасную функцию взятия головы списка: на пустом списке возвращайте **Nothing**.

```
safeHead :: [a] -> Maybe a
safeHead xs = case xs of
  [] -> ___
  x : _ -> ___
```

 **Решение.**

```
[] -> Nothing
x : _ -> Just x
```

# Разворот: где скрыта лишняя работа?

```
reverseSlow [] = []  
reverseSlow (x : xs) = reverseSlow xs ++ [x]
```

 **Устно:** Что делает  $(++)$  при каждом возврате? Какая сложность у разворота списка длины  $n$ ?

 **Решение.**

Обходит левый список: суммарно  $0 + 1 + \dots + (n - 1)$  операций, то есть  $O(n^2)$ .

# Аккумулятор из П4 возвращается

```
reverseFast xs = go [] xs
  where
    go acc [] = acc
    go acc (x : rest) = go (x : acc) rest
```

```
go [] [1,2,3]
⇒ go [1] [2,3]
⇒ go [2,1] [3]
⇒ go [3,2,1] []
⇒ [3,2,1]
```

- Добавление в начало —  $O(1)$ ; весь разворот —  $O(n)$

# Проектирование данных

# Запись: имена полей и обновление

```
data CatRecord = CatRecord { catName :: String, catAge :: Int }  
birthday cat = cat { catAge = catAge cat + 1 }
```

✎ **Устно:** Для `cat = CatRecord "murka"` 2 чему равны `catAge (birthday cat)` и `catAge cat`?

⚡ **Решение.**

3 и 2: обновление возвращает новую запись.

- Селектор поля — функция: `catAge :: CatRecord -> Int`
- **NB** Если поле есть не у всех конструкторов, селектор может быть частичным

# type или newtype?

## Синонимы

```
type ModuleId = Int
type CourseId = Int
```

Оба имени обозначают тот же тип `Int`.

## Новые типы

```
newtype ModuleId = ModuleId Int
newtype CourseId = CourseId Int
```

Компилятор различает идентификаторы.

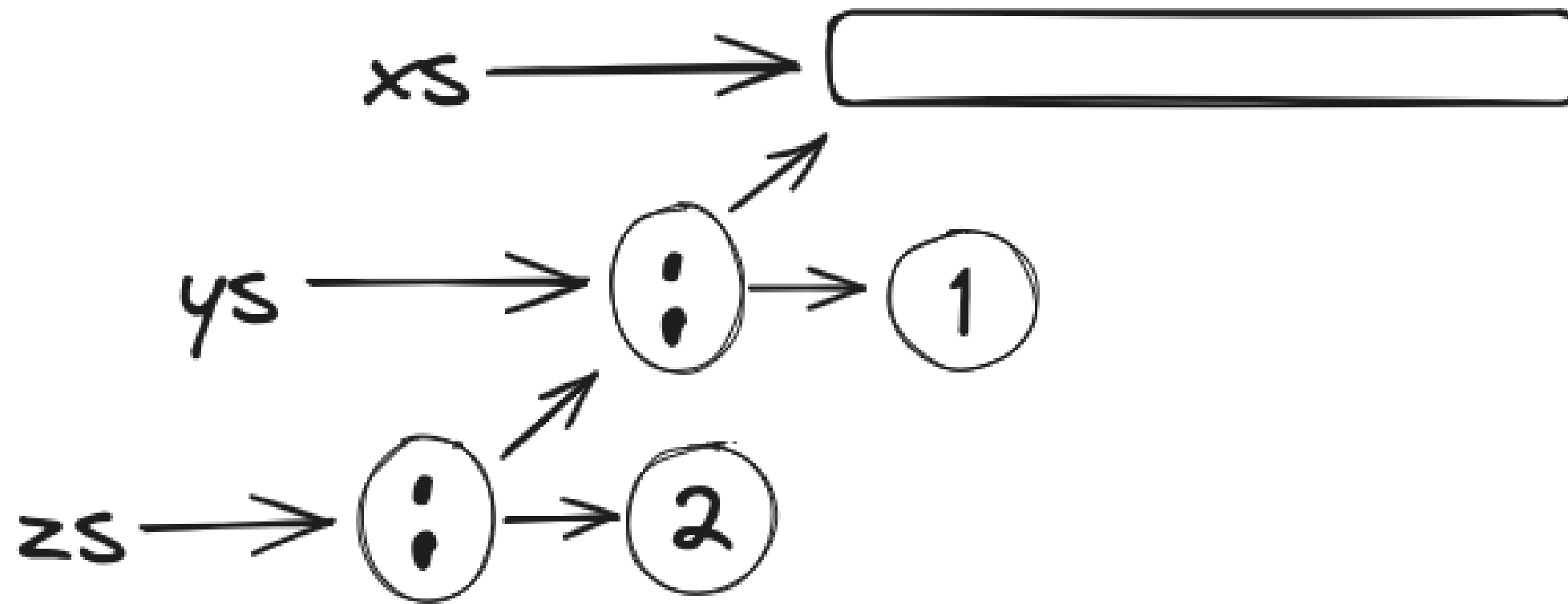
✎ **Устно:** Нужно запретить перестановку аргументов `publish :: ModuleId -> CourseId -> String`.  
Какой вариант подходит?

⚡ **Решение.**

`newtype`: ошибку представления можно поймать по типам до запуска.

# Персистентность: старый список остаётся

```
ys = 1 : xs  
zs = 2 : ys
```



- `xs`, `ys`, `zs` используют общий хвост
- Чтобы добавить голову, достаточно новой ячейки; хвост копировать не нужно

# Полнота и структурная рекурсия

| Проверяем                            | Пример из пары                                         |
|--------------------------------------|--------------------------------------------------------|
| Учтены все конструкторы              | <code>Parrot</code> получил ветку <code>Nothing</code> |
| Есть база рекурсии                   | <code>height (Leaf _) = 1</code>                       |
| Вызов идёт на меньшую структуру      | <code>height l, height r</code>                        |
| Результат выражает отсутствие данных | <code>Maybe Int, Maybe a</code>                        |

Для конечного дерева такая рекурсия завершается. Полнота сопоставления сама по себе завершение не гарантирует.

# Что вы теперь умеете

- Читать типы конструкторов и частично применять их
- Строить сопоставление по вариантам типа
- Писать рекурсию по дереву и списку
- Различать терм, тип и кайнд
- Выбирать `Maybe`, `type` и `newtype` по смыслу задачи

# Домашка: перенос техники

- Сначала выпишите конструкторы и варианты входа
- Затем базовые случаи и рекурсивные вызовы
- Проверьте пустые структуры и отсутствие данных
- Сверяйтесь с сигнатурой и запускайте `make check`

На следующей паре — общие интерфейсы разных типов: классы типов.

# Материалы

# Если тип применили неправильно

```
ghci> :kind Maybe Maybe
<interactive>:1:7: error: [GHC-83865]
• Expecting one more argument to ‘Maybe’
  Expected a type, but ‘Maybe’ has kind ‘* -> *’
• In the first argument of ‘Maybe’, namely ‘Maybe’
  In the type ‘Maybe Maybe’
```

`Maybe` ждёт обычный тип, а ему передали конструктор типа.

# Питомцы: ещё одно упражнение

Добавьте префикс "baron " к имени питомца.

```
baronify :: Pet -> Pet
baronify pet = case pet of
  Parrot -> Parrot
  Dog name years -> Dog ("baron " ++ name) years
  Cat (CatD name years) -> Cat (CatD ("baron " ++ name) years)
```

# Дублицирование элементов списка

Дублицируйте элементы с чётными номерами, начиная с нуля.

```
dupEvens :: [a] -> [a]
dupEvens = concat . map f . zip [0..]
  where
    f (i, x) = if odd i then [x] else [x, x]
```

# Кайнды: дополнительные примеры

```
data Free f a = Pure a | Free (f (Free f a))  
-- Free :: (* -> *) -> * -> *  
  
data HFree f a = HPure a | HFree (f (HFree f) a)  
-- HFree :: ((* -> *) -> * -> *) -> * -> *
```

# Полиморфные типы и система $\lambda\omega$

```
type Pair a b = forall c. (a -> b -> c) -> c
```

$\text{Pair Int Char} \equiv \text{forall } c. (\text{Int} \rightarrow \text{Char} \rightarrow c) \rightarrow c$

**Pair** :  $* \rightarrow * \rightarrow *$

**Pair** =  $\lambda\alpha^* \beta^*. \forall\gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$

**pair** :  $\forall\alpha \beta. \alpha \rightarrow \beta \rightarrow \mathbf{Pair} \alpha \beta$

**pair** =  $\Lambda\alpha^* \beta^*. \lambda x^\alpha y^\beta. (\Lambda\gamma^*. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y)$

**fst** :  $\forall\alpha \beta. \mathbf{Pair} \alpha \beta \rightarrow \alpha$

**fst** =  $\Lambda\alpha^* \beta^*. \lambda p^{\mathbf{Pair} \alpha \beta}. p \ \alpha \ (\mathbf{K} \ \alpha \ \beta)$

# Records и coerce: подробности

- Селекторы полей могут быть частичными у типов-сумм
- Для полей используют префиксы; дополнительные варианты синтаксиса дают расширения GHC
- `newtype` позволяет разделить типы без дополнительной обёртки во время исполнения

```
import Data.Coerce (coerce)
intsToIds :: [Int] -> [CourseId]
intsToIds = coerce
```

Расширения records · Parse, don't validate

# Типы-произведения в C++

```
#include <string>
struct CatT {
    std::string name;
    int age;
};
CatT createBarsik() { return CatT{"barsik", 4}; }
std::string getName(const CatT& cat) { return cat.name; }
```

Поля — компоненты произведения; конструктор создаёт значение, доступ к полю извлекает компонент.

# Типы-суммы в C

```
struct Pet {
    enum { CAT, DOG, PARROT } tag;
    union {
        struct { const char *name; int age; } cat;
        struct { const char *name; int age; } dog;
    } value;
};
/* tag и value должны согласовываться */
switch (pet.tag) {
case CAT:    printf("Cat: %s", pet.value.cat.name); break;
case DOG:    printf("Dog: %s", pet.value.dog.name); break;
case PARROT: printf("Parrot"); break;
}
```

# Типы-суммы в C++

```
std::variant<Cat, Dog, Parrot> pet{Cat{"barsik", 4}};  
if (std::holds_alternative<Cat>(pet)) {  
    std::cout << "Cat " << std::get<Cat>(pet).name;  
} else if (std::holds_alternative<Dog>(pet)) {  
    std::cout << "Dog " << std::get<Dog>(pet).name;  
} else if (std::holds_alternative<Parrot>(pet)) {  
    std::cout << "Parrot";  
}
```

std::variant

# Типы-суммы в Python

```
class Pet: pass
class Cat(Pet):
    def __init__(self, name): self.name = name
class Dog(Pet):
    def __init__(self, name): self.name = name
class Parrot(Pet): pass

if isinstance(pet, Cat):
    print("Cat " + pet.name)
elif isinstance(pet, Dog):
    print("Dog " + pet.name)
elif isinstance(pet, Parrot):
    print("Parrot")
else:
    raise AssertionError("unreachable code") # No exhaustiveness check
```

# Типы-суммы в Kotlin

```
sealed class Pet
class Cat(val name: String) : Pet()
class Dog(val name: String) : Pet()
class Parrot : Pet()

fun showPet(pet: Pet): String = when (pet) {
    is Cat -> "Cat: " + pet.name
    is Dog -> "Dog: " + pet.name
    is Parrot -> "Parrot"
}
```

Иерархия `sealed` позволяет компилятору проверять полноту выражения `when`.

# Что посмотреть в транспорте

- Parse, don't validate
- (youtube) Тагир Валеев — Pattern matching в Java и его воображаемые друзья
- The algebra (and calculus!) of algebraic data types